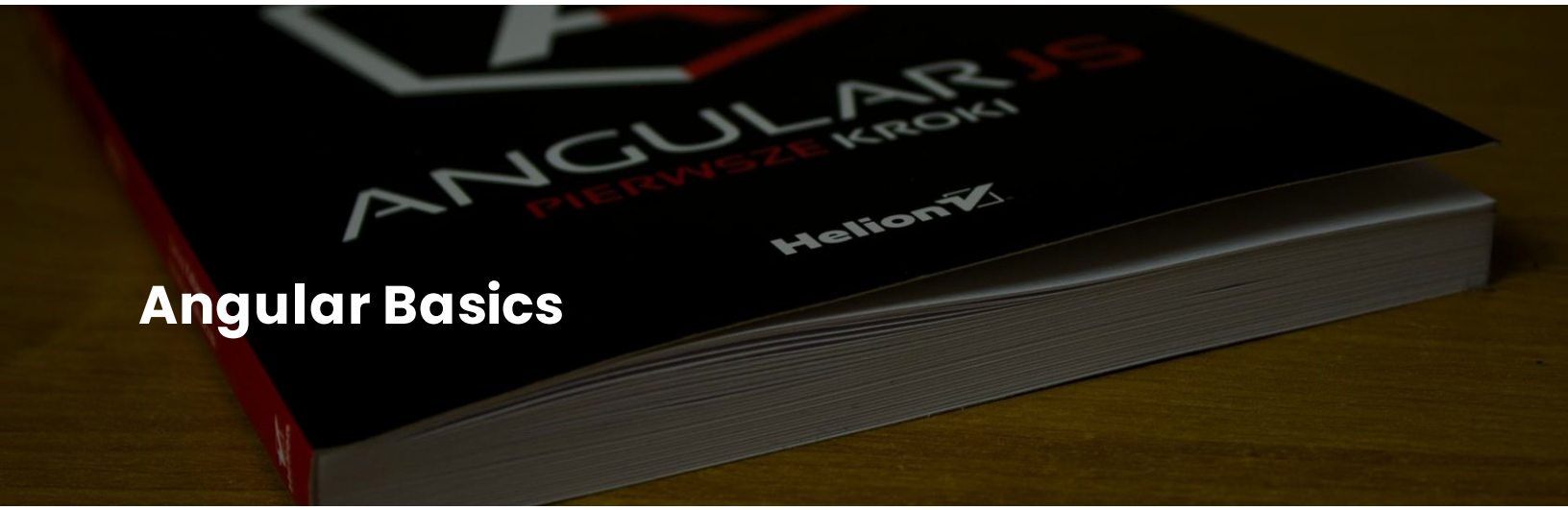


Angular Basics

Angular



Angular Basics

En este primer fastbook, veremos los conceptos básicos de Angular, desde cómo iniciar un proyecto en Angular y la preparación de nuestra máquina hasta los elementos básicos que nos ayudarán a generar una aplicación.

Nos centraremos a lo largo de todos los temas que configuran la asignatura en un mismo ejemplo que iremos evolucionando hasta que tengamos una web app real y totalmente funcional.

A este ejemplo de aplicación lo llamaremos Tea Minder, una aplicación con todo el sistema de autenticación (login, registro, etc.), así como hacer un CRUD (*Create*, *Read*, *Update*, *Delete*) para un Té.

Autor: Rodrigo Zavala

¿Qué es Angular?

TypeScript

Configurando mi workspace

Angular CLI

Angular Component

Component Life Cycle

Lección 1 de 6

¿Qué es Angular?



Angular es un framework de desarrollo web creado por Google.

Un framework es un **conjunto de herramientas y reglas** que facilitan la construcción de aplicaciones de software. Angular proporciona una estructura sólida y coherente para desarrollar aplicaciones web de una manera más rápida y organizada.

- [ ] En lugar de escribir todo el código desde cero, Angular te permite usar componentes reutilizables. Puedes crear un componente para el encabezado, otro para el pie de página y muchos otros para diferentes partes de tu aplicación.

- [ ] Además, Angular maneja tareas comunes automáticamente, como actualizar la interfaz cuando los datos cambian, gestionar la navegación entre páginas y manejar la comunicación con servidores. Esto ahorra tiempo y reduce la posibilidad de cometer errores.

- [ ] Al estar bajo el soporte de Google, es un framework muy robusto que se usa de forma muy habitual en entornos empresariales, además tiene mejoras constantes en forma de actualizaciones.

Historia de Angular



https://www.calibraint.com/blog/the-evolution-of-angular-versions

AngularJS (2009-2016)

El proyecto Angular se inició en 2009 por Misko Hevery y Adam Abrons en Google. En 2010, fue presentado al público como AngularJS. Fue uno de los primeros frameworks en popularizar el enfoque de desarrollo web basado en el lado del cliente, donde gran parte de la lógica y la interacción del usuario se manejaban en el navegador. AngularJS permitió a los desarrolladores crear aplicaciones web más dinámicas y ricas en características. Sin embargo, con el tiempo, surgieron desafíos en términos de rendimiento y mantenibilidad debido a la arquitectura del framework.

Angular 2+ (2016-actualidad)

En 2016, Google lanzó una reescritura completa de Angular conocida como Angular 2. Fue un cambio radical desde AngularJS y fue diseñado para abordar las limitaciones y desafíos de la primera versión. Angular 2 introdujo una arquitectura más modular y una forma más eficiente de manejar componentes y datos. Se eliminaron las limitaciones de rendimiento y se mejoró la escalabilidad. Desde entonces, Angular ha continuado evolucionando con nuevas versiones, introduciendo características y mejoras.

Angular CLI (2016-actualidad)

Junto con el lanzamiento de Angular 2, se presentó el Angular CLI (*Command Line Interface*). Este conjunto de herramientas de línea de comandos simplifica el proceso de configuración, creación y administración de proyectos Angular. Facilita la generación de componentes, módulos, servicios y otros elementos esenciales de una aplicación.

Angular Ivy (2019)

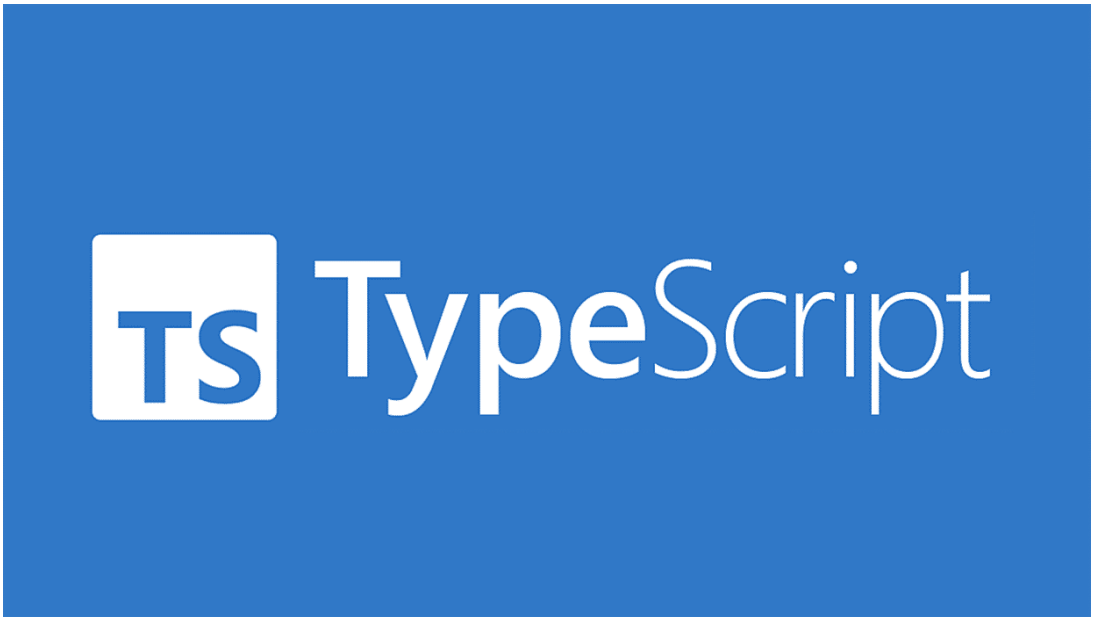
En 2019, Angular introdujo una actualización importante llamada Ivy: un nuevo motor de renderizado y compilación que reemplaza la antigua tecnología de Angular. Proporciona un rendimiento mejorado, un tamaño de paquete reducido y una mejor depuración. Ivy también permite un desarrollo más rápido y la creación de bibliotecas compartidas más eficientes.

Angular 14+ (2022-actualidad)

En las últimas versiones de Angular, se han añadido muchas mejoras, una de ellas, que ha tenido mucha relevancia son los *Standalone Components*, esto cambia el paradigma de Angular, ya que ahora tenemos componentes totalmente independientes sin necesidad de usar módulos, lo que vuelve a aplicaciones Angular en Moduless (más adelante, abordaremos este tema y el futuro de Angular).

Lección 2 de 6

TypeScript



Si hablamos de Angular, estamos obligados a hablar de TypeScript, ya que es el lenguaje base desde la versión 2 de Angular.

TypeScript viene a resolver algunos inconvenientes que presenta JavaScript, aquí veremos los puntos más interesantes.

1. **Tipado estático**: TypeScript es un lenguaje de programación que permite asignar tipos de datos a las variables. Esto significa que debes especificar qué tipo de datos contendrá una variable (como número, cadena, booleano, etc.) cuando la declaras. También ayuda a detectar errores de tipo en tiempo de compilación, en lugar de en tiempo de ejecución, lo que hace que el código sea más seguro.

2. **Superset de JavaScript**: TypeScript es un superset de JavaScript, lo que significa que cualquier código JavaScript válido también es código TypeScript. Puedes comenzar a usar TypeScript en proyectos existentes sin necesidad de realizar cambios drásticos.

3. **Interfaces y tipos personalizados**: TypeScript permite definir tipos de datos personalizados a través de interfaces y tipos. Las interfaces especifican la forma que debe tener un objeto, mientras que los tipos pueden ser más flexibles. Esto facilita la creación de estructuras de datos complejas y la definición de contratos claros entre las partes del código.

4. **Inferencia de tipos**: aunque debes especificar tipos en muchas ocasiones, TypeScript también puede inferir automáticamente los tipos cuando sea posible. Esto significa que no siempre necesitas escribir tipos explícitos si TypeScript puede deducirlos a partir de tu código.

5. **Compilación**: TypeScript no se ejecuta directamente en el navegador o en un entorno de ejecución JavaScript. En su lugar, se debe compilar el código TypeScript a JavaScript antes de que pueda ser ejecutado. Esto se hace con el compilador de TypeScript (tsc), que traduce el código TypeScript a JavaScript para que cualquier navegador o el motor JavaScript lo pueda entender.

6. **Soporte para ES6 y posteriores**: este lenguaje también ofrece compatibilidad total con ECMAScript 6 (ES6) y versiones posteriores de JavaScript. Puedes utilizar las características más modernas de JavaScript, como clases, módulos y funciones de flecha, junto con las ventajas del tipado estático.

7 **Ecosistema y herramientas**: TypeScript tiene un ecosistema en crecimiento con una gran comunidad de desarrolladores. Además, se integra bien con herramientas populares como Visual Studio Code, lo que facilita la escritura y depuración de código TypeScript.

8 En definitiva, TypeScript es un **lenguaje de programación que agrega tipado estático a JavaScript**, lo que mejora la seguridad y el mantenimiento del código. También proporciona otras características útiles, como interfaces, tipos personalizados y compatibilidad con las últimas características de JavaScript.

Para mejorar tu aprendizaje te recomiendo que consultes la documentación oficial de TypeScriptlang.

Lección 3 de 6

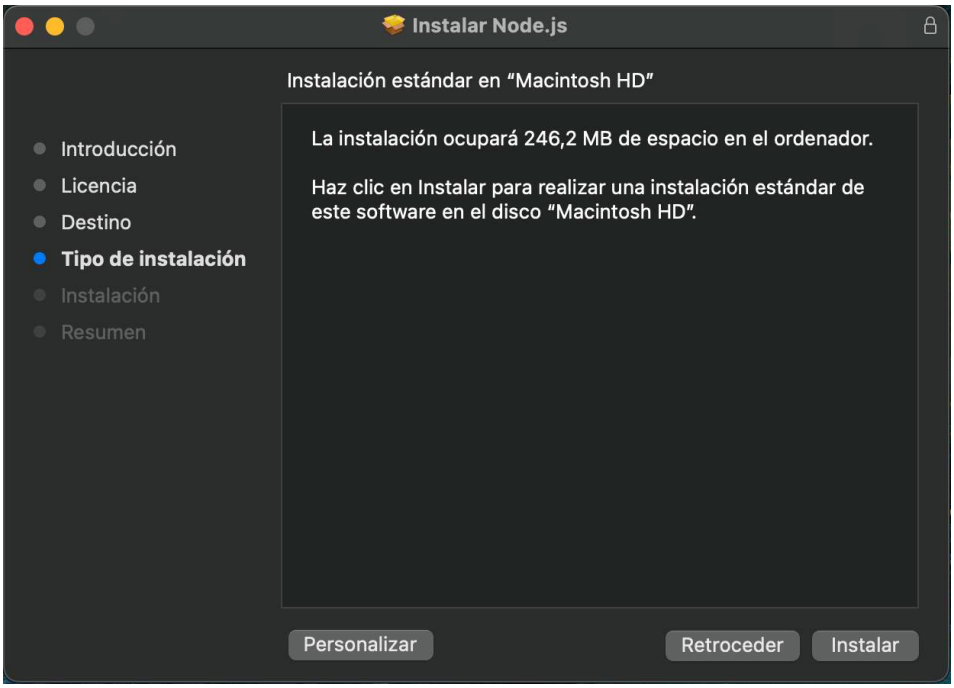
Configurando mi workspace

Antes de poder crear nuestra primera aplicación en Angular, necesitamos una serie de programas y herramientas, en este bloque vamos a ver qué necesitamos y cómo instalarlos.

Node.js

Para empezar, necesitamos **instalar la última versión de Node.js**, esto lo podemos hacer desde la página oficial https://nodejs.org/es, **siempre descargaremos la versión LTS** (*long term support*).

El proceso de instalación es sencillo, el instalador nos hará un paso a paso que debemos ir siguiendo.





Ahora, para **comprobar que se ha instalado correctamente** dentro de nuestra consola lanzaremos el comando *node -v* y nos tiene que devolver algo similar a esto:

NPM (*Node Package Manager*)

Es el sistema de gestión de paquetes para Node.js y su ecosistema. En términos simples, es una herramienta que nos permite **instalar, gestionar y compartir piezas de código reutilizable** (llamadas paquetes o módulos) escritas en JavaScript. Estos paquetes pueden contener funciones, librerías y componentes que facilitan la construcción de aplicaciones.

NPM nos permite realizar estas **funciones**:

Instalar paquetes

Puedes instalar paquetes desde el registro público de NPM utilizando el comando npm *install nombre-del-paquete*.

Administrar dependencias

Las aplicaciones suelen depender de varios paquetes. NPM administra automáticamente estas dependencias y sus versiones para asegurarse de que todo funcione correctamente.

Gestionar versiones

Puedes especificar las versiones exactas o rangos de versiones de los paquetes que deseas utilizar en tu proyecto. Esto es importante para mantener la consistencia y evitar problemas de compatibilidad.

Publicar y compartir paquetes

Si desarrollas una funcionalidad útil, puedes empaquetarla en un paquete NPM y compartirlo con otros desarrolladores de todo el mundo.

Ejecutar scripts personalizados

Puedes definir scripts personalizados en el archivo 'package.json' para automatizar tareas comunes, como la ejecución de pruebas, la construcción de tu aplicación...

Y una vez tengamos Node.js instalado, también **tendremos su gestor de paquetes NPM**, ya que viene incluida en la instalación.

Terminal

Para poder ejecutar comandos **necesitaremos una terminal**, puedes usar cualquiera, incluso hay IDEs o editores de código como VS Code, del que hablaremos más adelante, que integran la terminal.

Pero como recomendación en el caso de MacOS usaremos iTerm (https://iterm2.com/).

iTerm es **la terminal por excelencia para los desarrolladores**, es muy personalizable y tiene muchos plugins que se pueden instalar y para Windows recomiendo Windows Terminal (Windows Store). Este último puede abrir distintas instancias de terminal como PowerShell, Git Bash, etc.

Lección 4 de 6

Angular CLI

La CLI de Angular, o interfaz de línea de comandos, es una potente herramienta que permite a los desarrolladores crear, gestionar y construir aplicaciones Angular desde la línea de comandos. La CLI proporciona una serie de comandos útiles para **generar componentes, servicios, módulos y otros archivos relacionados con Angular**, así como para construir y servir aplicaciones.

Estos son algunos de los **comandos más utilizados en la CLI de Angular**, ¡toma nota!

ng new

Crea un nuevo proyecto Angular en un nuevo directorio.

ng generate

Genera nuevo código Angular, como componentes, servicios y módulos.

ng serve

Inicia un servidor de desarrollo local, que sirve la aplicación Angular y proporciona recarga en vivo a medida que se realizan cambios en el código.

ng build

Construye la aplicación Angular para producción, generando un conjunto de archivos estáticos que pueden ser servidos desde un servidor web.

ng test

Ejecuta pruebas unitarias para la aplicación Angular.

ng e2e

Ejecuta pruebas de extremo a extremo para la aplicación Angular.

Además de estos comandos, la CLI de Angular también proporciona una serie de opciones para personalizar el comportamiento de los comandos y el código generado.

La CLI de Angular es una herramienta esencial para cualquier desarrollador que trabaje con Angular, simplifica muchas de las tareas comunes asociadas con la creación y gestión de aplicaciones Angular.

Para instalar Angular CLI, una vez ya tengamos nuestro entorno de trabajo preparado, solo tenemos que lanzar el siguiente comando *npm install -g @angular/cli*, **este comando nos instalará el CLI de Angular** a nivel global (-g) y al no especificar la versión del mismo, se instalará la última versión estable.

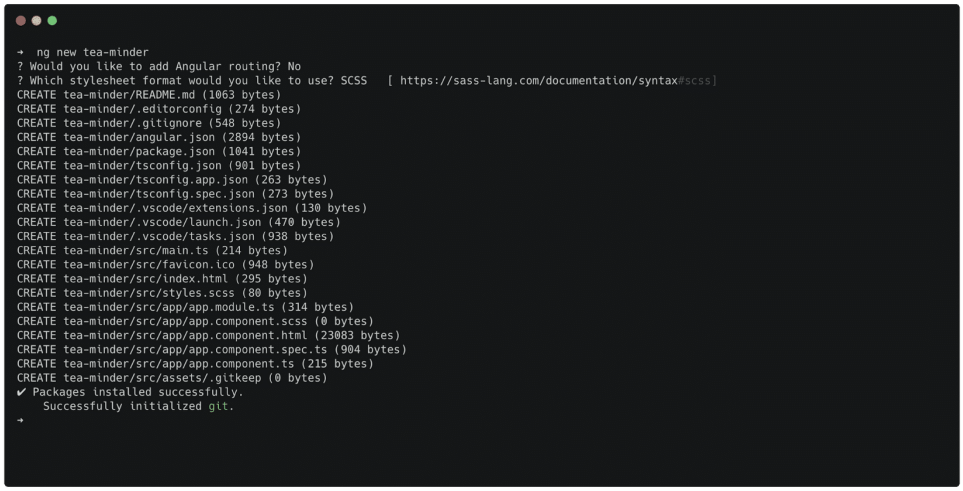


```
➜  ~ npm install -g @angular/cli

added 244 packages in 19s

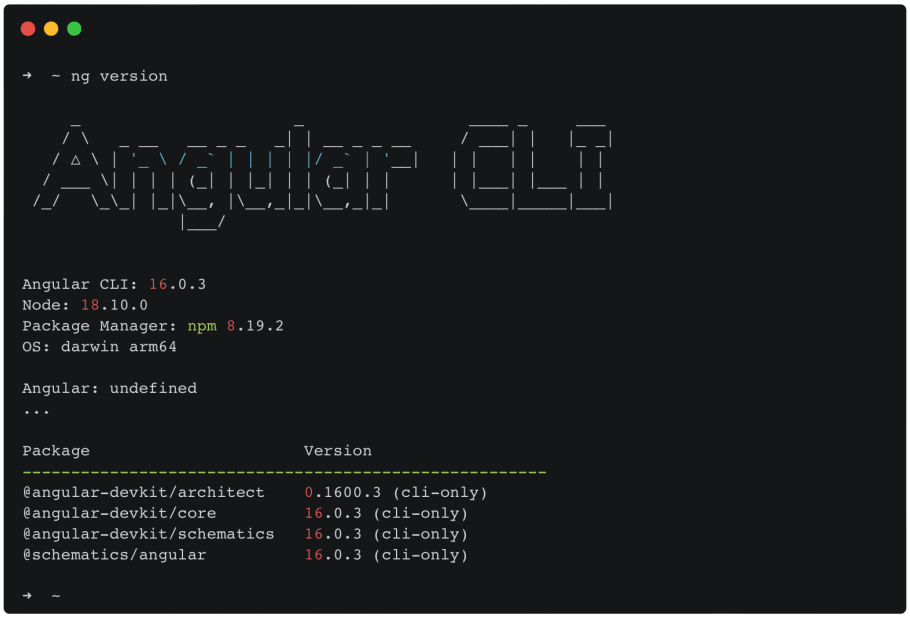
36 packages are looking for funding
  run `npm fund` for details
➜  ~
```

Una vez finalice la instalación podemos **lanzar el comando *ng version***, este comando nos devolverá la versión instalada de Angular.



```
➜  ng new tea-minder
? Would you like to add Angular routing? No
? Which stylesheet format would you like to use? SCSS   [ https://sass-lang.com/documentation/syntax#scss]
CREATE tea-minder/README.md (1063 bytes)
CREATE tea-minder/.editorconfig (274 bytes)
CREATE tea-minder/.gitignore (548 bytes)
CREATE tea-minder/angular.json (2894 bytes)
CREATE tea-minder/package.json (1041 bytes)
CREATE tea-minder/tsconfig.json (901 bytes)
CREATE tea-minder/tsconfig.app.json (263 bytes)
CREATE tea-minder/tsconfig.spec.json (273 bytes)
CREATE tea-minder/.vscode/extensions.json (130 bytes)
CREATE tea-minder/.vscode/launch.json (470 bytes)
CREATE tea-minder/.vscode/tasks.json (938 bytes)
CREATE tea-minder/src/main.ts (214 bytes)
CREATE tea-minder/src/favicon.ico (948 bytes)
CREATE tea-minder/src/index.html (295 bytes)
CREATE tea-minder/src/styles.scss (80 bytes)
CREATE tea-minder/src/app/app.module.ts (314 bytes)
CREATE tea-minder/src/app/app.component.scss (0 bytes)
CREATE tea-minder/src/app/app.component.html (23083 bytes)
CREATE tea-minder/src/app/app.component.spec.ts (904 bytes)
CREATE tea-minder/src/app/app.component.ts (215 bytes)
CREATE tea-minder/src/assets/.gitkeep (0 bytes)
✔ Packages installed successfully.
    Successfully initialized git.
➜
```

Aunque a lo largo de la asignatura iremos viendo los distintos comandos que existen para crear todo tipo de elementos de Angular, ya puedes consultar todos los comandos existentes en la documentación official de Angular: https://angular.io/cli.



```
➜  ~ ng version

     _                      _                 ____ _     ___
    / \   _ __   __ _ _   _| | __ _ _ __     / ___| |   |_ _|
   / △ \ | '_ \ / _` | | | | |/ _` | '__|   | |   | |    | |
  / ___ \| | | | (_| | |_| | | (_| | |      | |___| |___ | |
 /_/   \_\_| |_|\__, |\__,_|_|\__,_|_|       \____|_____|___|
                |___/


Angular CLI: 16.0.3
Node: 18.10.0
Package Manager: npm 8.19.2
OS: darwin arm64

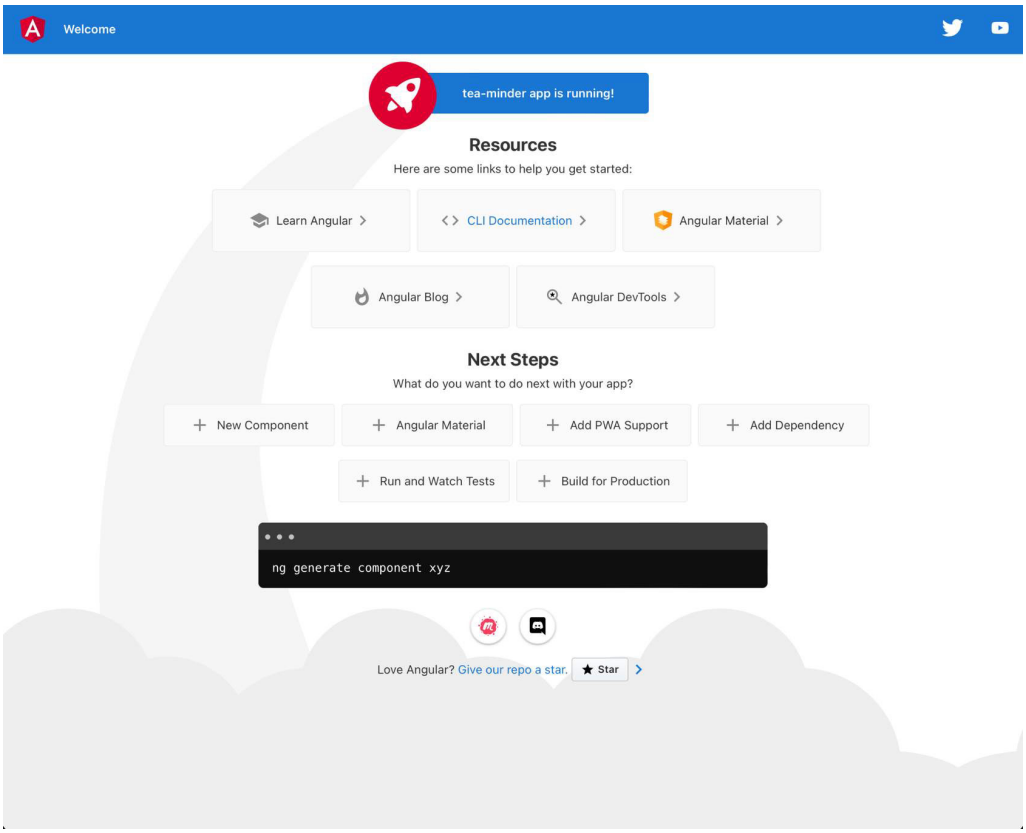
Angular: undefined
...

Package                      Version
------------------------------------------------------
@angular-devkit/architect    0.1600.3 (cli-only)
@angular-devkit/core         16.0.3 (cli-only)
@angular-devkit/schematics   16.0.3 (cli-only)
@schematics/angular          16.0.3 (cli-only)

➜  ~
```

Ahora que tenemos el CLI de Angular, podemos **crear una nueva aplicación**. Para ello, usaremos el comando ng new, crearemos un nuevo proyecto y lo llamaremos *tea-minder*.

Nosotros usaremos el formato SCSS para los estilos, aunque, de momento, no incluiremos *routing*. Después, dentro de la carpeta que nos ha creado *tea-minder*, lanzaremos el comando *ng serve*, ya que nos permite iniciar el servidor de Angular en local. Pues bien, una vez se haya compilado nuestra aplicación, podremos ir a nuestro navegador, acceder a la URL http://localhost:4200 y… ¡ver nuestra aplicación funcionando!



Componentes principales de Angular

Estos son los **diferentes tipos de componentes** que se pueden generar utilizando el comando *ng generate* en Angular CLI. Cada tipo de componente tiene su propio caso de uso único en una aplicación Angular y se puede personalizar aún más para adaptarse a nuestras necesidades.

1 **Services**

Un servicio es una forma de compartir datos o funcionalidad entre diferentes componentes en una aplicación Angular.

El comando *ng generate service*, por ejemplo, se utiliza para crear un nuevo servicio.

```
ng generate service my-service
```

Esto generará un nuevo servicio llamado *my-service* en el directorio *src/app/*, con su archivo TypeScript asociado.

```
import { Injectable } from '@angular/core';
import { Subject } from 'rxjs';

@Injectable()
export class DataService {
  private sharedData = new Subject<string>();
  sharedData$ = this.sharedData.asObservable();

  sendData(data: string) {
    this.sharedData.next(data);
  }
}
```

data.service.ts

2 Pipes

Un pipe es una forma de transformar los datos antes de mostrarlos en la vista.

El comando *ng generate pipe* se utiliza para crear un nuevo pipe.

```
ng generate pipe my-pipe
```

Esto **generará un nuevo pipe** llamado *my-pipe* en el directorio *src/app/*, con su archivo TypeScript asociado.

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'myPipe'
})
export class MyPipePipe implements PipeTransform {
  transform(value: string): string {
    return value.toUpperCase();
  }
}
```

3 Directives

Una directiva es una forma de agregar comportamiento a un elemento HTML existente o crear un nuevo elemento HTML reutilizable.

Y es el comando *ng generate directive* el que se utiliza para crear una nueva directiva.

```
ng generate directive my-directive
```

Esto generará una nueva directiva llamada *my-directive* en el directorio *src/app/*, con su archivo TypeScript asociado.

```
import { Directive, ElementRef } from '@angular/core';

@Directive({
  selector: '[appMyDirective]'
})
export class MyDirectiveDirective {
  constructor(private el: ElementRef) {
    el.nativeElement.style.backgroundColor = 'yellow';
  }
}
```

4 **Inyección de dependencias**

La inyección de dependencias es un concepto clave en Angular que nos permite proporcionar instancias de clases necesarias a los componentes.

Imagina que estamos construyendo una aplicación que muestra una lista de productos. Queremos crear un servicio que nos ayude a obtener estos productos desde una fuente externa, como una API. La inyección de dependencias **nos permite conectar el componente con el servicio**, de modo que el componente pueda acceder a los datos de productos proporcionados por el servicio.



```
import { Component } from '@angular/core';
import { ProductService } from './product.service';

@Component({
  selector: 'app-product',
  template: '<ul><li *ngFor="let product of products">{{ product }}</li></ul>'
})
export class ProductComponent {
  products: string[] = [];

  constructor(private productService: ProductService) {
    this.products = this.productService.getProducts();
  }
}
```

product.component.ts

Si analizamos este ejemplo, podemos ver que:

- Hemos creado un servicio llamado **ProductService**, que tiene un método ***getProducts()*** que devuelve una lista de productos.
- Nuestro componente **ProductComponent** utiliza la inyección de dependencias para obtener una instancia del **ProductService** en su constructor.

- [ ] En el constructor del componente, llamamos al método ***getProducts()*** del servicio y asignamos la lista de productos al arreglo *products*. Luego, utilizamos este arreglo en la plantilla (*template*) para mostrar los productos en una lista.

La inyección de dependencias en Angular nos permite mantener nuestro código organizado, promoviendo la reutilización y el mantenimiento eficiente de la aplicación. Nos ayuda a conectar componentes y servicios de manera fluida y coherente.

Angular Component

Un componente es **un bloque de construcción básico de una aplicación Angular**. Consiste en una clase TypeScript y una plantilla HTML. El comando *ng generate component* se utiliza para crear un nuevo componente.

```
ng generate component my-component
```

Esto generará un nuevo componente llamado *my-component* en el directorio *src/app/*, con sus archivos asociados HTML, SCSS y TypeScript.



Two-way Binding

El two-way binding o doble enlace es una característica en Angular que nos permite mantener sincronizados automáticamente los datos en la interfaz de usuario (HTML) y en el componente subyacente (Typescript). Esto significa que **cuando los datos cambian en la interfaz de usuario o plantilla, también se actualizan en el componente y viceversa**.

Por ejemplo, si hay un cuadro de texto en la interfaz de usuario y se escribe algo en él, el *two-way binding* garantiza que lo que se escriba se refleje al instante en una variable dentro del código Angular, en el componente que corresponde. Del mismo modo, si se cambia esa variable en el componente, el texto en el cuadro correspondiente se actualizará automáticamente.



```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  mensaje: string = ''; // Variable para almacenar el mensaje

  constructor() {}
}
```

app.component.ts

Para implementar el *two-way binding* en Angular, generalmente, se utiliza la directiva [(ngModel)]. Esta directiva **combina la propiedad de enlace de datos y el evento de cambio en un solo paso**, permitiendo que los cambios en la interfaz de usuario se reflejen automáticamente en el componente y viceversa.



```
<input [(ngModel)]="mensaje" placeholder="Escribe un mensaje">
<p>El mensaje es: {{ mensaje }}</p>
```

app.component.html

Por lo tanto, recuerda siempre que el *two-way binding* en Angular es una forma eficiente y conveniente de mantener **sincronizados los datos entre la interfaz de usuario** y el componente, asegurando que cualquier cambio en uno de ellos se refleje automáticamente en el otro.

Comunicación entre componentes

Las aplicaciones Angular se basa en la modularización y la reutilización de sus componentes, para esto debemos tener en cuenta que debemos siempre intentar hacer los componentes con una única responsabilidad. Y si es un componente muy complejo **lo ideal es dividirlo y hacer componentes más pequeños y que se puedan mantener**, con esto además conseguiremos tener componentes pequeños para poder reutilizarlos si fuera necesario.

Como Angular se basa en módulos y componentes, debemos saber cómo podemos comunicar la información entre ellos. Ahora, veremos los casos más típicos de comunicación entre componentes, dependiendo de la relación que exista entre ambos.

1 De hijo a padre

Imagina que tienes un componente hijo que necesita enviar información al componente padre. Esto se llama comunicación de 'hijo a padre'. Puedes lograrlo emitiendo un evento desde **el componente hijo y escuchándolo en el componente padre**. Es como si el hijo levantara la mano y el padre respondiera. Esto permite que el componente hijo comunique datos o eventos al componente padre que lo contiene.

```typescript
import { Component, Output, EventEmitter } from '@angular/core';

@Component({
  selector: 'app-child',
  template: '<button (click)="emitData()">Enviar datos al padre</button>'
})
export class ChildComponent {
  @Output() dataEmitted = new EventEmitter<string>();

  emitData() {
    this.dataEmitted.emit('¡Hola desde el hijo!');
  }
}
```

child.component.ts

```typescript
import { Component } from '@angular/core';

@Component({
  selector: 'app-parent',
  template: '<app-child (dataEmitted)="handleData($event)"></app-child><p>{{ receivedData }}</p>'
})
export class ParentComponent {
  receivedData: string = '';

  handleData(data: string) {
    this.receivedData = data;
  }
}
```

parent.component.ts

2 De padre a hijo

Ahora, supongamos que necesitamos enviar datos del componente padre al hijo. Podemos lograrlo pasando datos a través de **propiedades de entrada** (*input properties*) en el componente hijo. Imaginemos que el padre le da algo al hijo antes de que este empiece a funcionar. Así, el hijo puede usar esos datos para mostrar información o realizar acciones.

```
import { Component, Input } from '@angular/core';

@Component({
  selector: 'app-child',
  template: '<p>{{ message }}</p>'
})
export class ChildComponent {
  @Input() message = '';
}
```

3 Entre hermanos (mediante un servicio)

A veces, tienes componentes que son hermanos, es decir, están al mismo nivel en la jerarquía. Para comunicar datos entre hermanos, una opción es utilizar un servicio compartido. Un servicio es como un mensajero en el medio. Un componente coloca datos en el servicio y el otro componente los toma. De esta forma, los componentes hermanos pueden compartir información sin que tengan una relación directa entre ellos.

```
import { Injectable } from '@angular/core';
import { Subject } from 'rxjs';

@Injectable()
export class DataService {
  private sharedData = new Subject<string>();
  sharedData$ = this.sharedData.asObservable();

  sendData(data: string) {
    this.sharedData.next(data);
  }
}
```

data.service.ts



```ts
import { Component } from '@angular/core';
import { DataService } from './data.service';

@Component({
  selector: 'app-sibling2',
  template: '<p>{{ receivedData }}</p>'
})
export class Sibling2Component {
  receivedData: string = '';

  constructor(private dataService: DataService) {
    this.dataService.sharedData$.subscribe(data => {
      this.receivedData = data;
    });
  }
}
```

sibling1.component.ts
sibling2.component.ts

Totalmente independientes

Por último, hay momentos en los que **los componentes necesitan comunicarse, pero no están conectados en la misma jerarquía**. Para estas situaciones, puedes utilizar servicios de eventos o bibliotecas externas. Estas herramientas permiten emitir y escuchar eventos en diferentes partes de tu aplicación, sin importar dónde estén ubicados los componentes.

Component Life Cycle

El ciclo de vida de un componente en Angular consta de varios eventos que ocurren desde su creación hasta su destrucción.

Aquí te comparto una breve descripción de los principales eventos en el ciclo de vida de un componente en Angular:

1. **ngOnChanges**: este evento se dispara cuando los datos de entrada (@Input) de un componente cambian.
2. **ngOnInit**: se ejecuta después de que Angular haya inicializado el componente. Es un buen lugar para realizar inicializaciones adicionales.
3. **ngDoCheck**: este evento se dispara cuando Angular realiza su detección de cambios. Puedes usarlo para realizar comprobaciones personalizadas.
4. **ngAfterContentInit**: ocurre después de que Angular haya proyectado el contenido en el componente.

5. **ngAfterContentChecked**: se dispara después de que Angular haya verificado el contenido proyectado.

6. **ngAfterViewInit:** este evento se ejecuta después de que Angular haya inicializado la vista del componente.

7. **ngAfterViewChecked**: se dispara después de que Angular haya verificado la vista del componente.

8. **ngOnDestroy**: es el último evento y se dispara justo antes de que el componente sea destruido. Es un buen lugar para realizar limpieza, como detener suscripciones y liberar recursos.

Estos eventos proporcionan **puntos de gancho en el ciclo de vida de un componente** que te permiten ejecutar código en momentos específicos, lo que es útil para realizar tareas como la inicialización, la actualización y la limpieza en una aplicación Angular.

Y después de revisar cómo es el ciclo de vida de un componente, damos por cerrado el primer tema sobre **los básicos de Angular**: esos conceptos y comandos imprescindibles para dar tus primeros pasos en este framework tan versátil. Pero antes de irte, no olvides explorar la documentación extra que te hemos compartido a lo largo de estas páginas.

¡Enhorabuena! Fastbook superado